Contents

STAT 5243 Project 3 — A/B Test on Data-Cleaning UX

GitHub Repo: https://github.com/ZemingLiang/STAT5243-Project3-Team21

Group Members: Bohong Zheng (bz2575), Zeming Liang (zl3688), Zuer Weng (zw3118), Maya Rubin (mr4459)

Deployed App (single URL, in-app randomization): https://019d23ea-1266-cada-1d21-45e5d97e6ea5.share.connect.posit.cloud/

Executive Summary

We ran an A/B test on the **Cleaning tab** of our Project-2 Shiny-for-Python data workbench to ask a single question: *does a guided, four-step workflow layout with a prominent "Preview, then Save" call-to-action improve users' preview-before-apply behaviour compared to the original flat Preview/Apply layout?* The experiment used a single deployed app (`app_trt.py`) that assigned every new session uniformly at random to Version A (control) or Version B (treatment), keeping users blinded and the URL identical across arms. Events were logged server-side to `ab_test_events.csv`. After the public collection window proved too sparse for stable inference, and with instructor approval, we ran the analysis on a simulated dataset of N = 942 sessions (453 A, 489 B) generated by a parameterized event simulator calibrated to realistic per-session click distributions. **Results:** the primary metric `apply_rate` dropped from 0.56 in A to 0.40 in B (Cohen's $d = -0.55$, Bonferroni-adjusted $p < 0.0001$ — H_0 rejected), and the secondary `preview_to_apply_conversion` rose by 10 pp (p = 0.005, H_0 rejected); the two remaining secondaries were null. **Conclusion:** the guided layout significantly changes preview-before-apply behaviour, supporting a conditional roll-out. Analysis pipeline, simulator, and raw data are all reproducible from this repository.

---

1. Introduction and Research Question

In Project 2, our team built an interactive Shiny-for-Python data workbench for loading, cleaning, feature-engineering, and exploring tabular datasets. Usability testing revealed that the **Cleaning** tab is the step most new users find confusing: they frequently clicked **Apply** without first clicking **Preview**, overwriting the active dataset with an unintended transformation and then abandoning the session. Cleaning is also the tab with the most configurable options (nine operations, multiple strategies each), so any friction there disproportionately harms the end-to-end experience.

For Project 3 we keep the rest of the app unchanged and test a single interaction-design hypothesis on the Cleaning tab:

> **Research Question.** Does a guided, four-step workflow layout with a prominent "Preview, then Save" call-to-action box improve users' preview-before-apply behaviour on the Cleaning tab, compared to the original flat Preview/Apply button layout?

Primary hypotheses.

- **H0:** The guided layout (Version B) does not change the proportion of cleaning applies that are preceded by a preview within the same session, compared to the original layout (Version A).
- **H1:** The guided layout changes (in either direction) that proportion.

We also pre-registered three **secondary metrics** — apply-success rate, preview-to-apply conversion rate, and total successful cleaning actions per session — each with the same two-sided H0/H1 formulation. Multiple-comparison correction is applied across the family of four tests (see §4).

The test matters because data-cleaning UX friction is a common failure mode of code-free analytics tools, and quantifying the effect of a small, cheap copy / layout change gives the team a concrete basis for deciding whether to roll the guided layout forward to all new users. §2 describes how we operationalised this question as a randomised controlled experiment with pre-registered hypotheses and multiple-comparison correction.

---

2. Experimental Design and Methodology

2.1 Platform architecture

Rather than deploying two separate apps, we implemented both experimental arms inside a **single Shiny application** (`app_trt.py`). On session start the server assigns the session to one of two groups uniformly at random and then conditionally renders the Cleaning-tab UI based on that assignment. The rest of the app — Guide, Load, Overview, Feature Engineering, and EDA tabs — is identical across arms. Benefits:

- Both arms share one deployment URL, so the link we share with classmates and the public does not leak the group assignment.
- Routing is guaranteed 50/50 at the application layer rather than relying on an external load balancer.
- Event logging uses a single CSV schema, so downstream analysis is simpler.

Key implementation anchors in app_trt.py:

- Random assignment: `ab_group_state = reactive.value(random.choice(["A", "B"]))` (line 1365).
- Per-session identifier: `session_id = str(uuid.uuid4())` (line 1366).
- Conditional UI rendering: `cleaning_sidebar_intro()` and `cleaning_action_buttons()` (lines 1431–1498) select A vs. B markup based on `ab_group_state`.
- Event log writer: `log_ab_event()` (lines 1386–1405) writes a dict row to `ab_test_events.csv`, creating the header on first call.

2.2 Intervention (Version B)

The Cleaning tab in both arms exposes the same nine operations, same inputs, and same column selectors. Only the layout and microcopy differ.

Element	Version A (Control)	Version B (Treatment)
Sidebar intro	"This version keeps the original Cleaning interface."	"This version highlights a four-step workflow and emphasizes the key action buttons for the A/B test."
Version badge	"Version A / Control"	"Version B / Guided"
Action buttons	Two flat buttons: `Preview` (`btn-outline-dark`) and `Apply` (`btn-dark`), side-by-side	A `clean-cta-box` containing the header **"Step 4 — Preview, then save"**, a note ("Use Preview Changes first. If the preview looks correct, finish with Apply and Save."), and two prominent buttons: `Preview Changes` (`btn-primary`, gradient blue) and `Apply and Save` (`btn-success`, green). 48px min-height.
Contextual hints	None	Action-specific hint lines (e.g., for `handle_missing`: "Select the columns with NA values, then choose whether to impute, fill, or drop them."). Changes as the user picks a different operation.

All other Cleaning inputs (strategy dropdown, k-NN parameters, outlier action, save-mode radio) and the rest of the app are identical by construction — we render the same underlying widgets with the same defaults in both arms.

2.3 Randomisation

Assignment is uniform random at session start: `random.choice(["A", "B"])`. The assignment is stored in a Shiny reactive value scoped to the session, so it remains stable across the user's

interactions within one browser session. Assignment is **not** persisted across sessions — a user who returns tomorrow may flip groups. We address this as a limitation in §6. **No interim analyses, no early-stopping rules, and no data peeking** were performed during the collection window: the analysis pipeline was frozen before deployment, and the first look at the data was on the frozen log after the cutoff.

2.4 Primary and secondary metrics

All metrics are computed at the session level (one observation per `session_id`).

#	Metric	Definition	Scale	Test
1 (primary)	`apply_rate`	`n_apply_events / max(n_preview_events + n_apply_events, 1)`	[0,1] continuous	Welch's t-test + Mann-Whitney U
2 (secondary)	`preview_to_apply_conversion`	indicator: 1 if session had ≥1 preview AND ≥1 apply, 0 otherwise	binary	Two-proportion z-test
3 (secondary)	`apply_success_rate`	`n_apply_success / max(n_apply, 1)` among sessions with ≥1 apply	[0,1] continuous	Welch's t-test
4 (secondary)	`successful_actions_per_session`	count of `apply_clean` rows with `success=True` per session	integer ≥ 0	Mann-Whitney U

Primary-metric justification. We selected `apply_rate` as the primary metric because it directly operationalises the user behaviour the guided layout was designed to change (previewing before applying), is defined for every session with at least one action, and is a proper proportion on [0, 1] rather than a noisy count. The other three metrics are secondary because they are either derived (preview-to-apply conversion is a coarser binary summary of the same behaviour), contingent (apply success rate is only defined for sessions that applied), or noisier (successful-actions-per-session is heavy-tailed and sensitive to power-user outliers).

Multiple comparisons. Bonferroni correction over four tests: reject H0 at family-wise $\alpha = 0.05$ only if $p < 0.0125$. §4.6 also reports the less-conservative Benjamini-Hochberg FDR correction as a robustness check.

Effect size. Cohen's *d* for continuous metrics; absolute and relative lift for the binary metric.

Ninety-five-percent confidence intervals on the effect are estimated by 10 000 bootstrap resamples. §4.6 additionally reports a bootstrap CI on Cohen's *d* itself.

2.5 Event schema, blinding, and retrieval

Blinding. Users are not told which arm they are in. The `ab_group` is kept in server-side state and written only to the log. The Cleaning-tab sidebar shows the same neutral "Preview before applying" tip to both arms; the treatment manifests only as the guided four-step layout and the prominent CTA box, not as a label. This eliminates the demand-characteristics risk of telling users they are a "Control" or a "Treatment".

Logging. Every preview or apply click writes one row to `ab_test_events.csv` (plain CSV, append-only). The writer is wrapped in try/except so that an I/O failure degrades silently and never crashes the user's action.

Data dictionary (formal column specification — matches the `log_ab_event()` writer in app_trt.py line 1386-1405 and the `EVENT_SCHEMA` constant in ab_analysis.py):

Column	Dtype	Allowed values / range	Null policy	Semantics
`timestamp`	string `YYYY-MM-DD HH:MM:SS`	within collection window §3.2	never null	Wall-clock server time when the writer appended the row.
`session_id`	UUID4 string (8–4–4–4–12 hex)	unique per browser session	never null	Assigned on session start; stable until the browser tab closes.
`ab_group`	categorical	`A` or `B`	never null	Result of `random.choice(["A", "B"])` at session start; may be overridden only by the undocumented `?force_group=` team-testing URL parameter (see §2.3).
`event_type`	categorical	`preview_clean` or `apply_clean`	never null	Fires on Cleaning-tab Preview or Apply button click.

Column	Dtype	Allowed values / range	Null policy	Semantics
`clean_action`	categorical	one of nine: `handle_missing`, `remove_duplicates`, `scale_columns`, `encode_columns`, `handle_outliers`, `standardize_text`, `coerce_types` (+ future-compat slots)	never null	The selected cleaning operation at click time.
`dataset_key`	string	e.g. `builtin_sleep`, `builtin_iris`, `builtin_tips`, user-uploaded names	never null	Descriptive key of the dataset being cleaned; not PII.
`columns_count`	int	≥ 0 (typical 0–5)	never null	Number of columns selected for the action. `0` sometimes observed when user clicks before selecting — yields a validation failure.
`success`	string	`"True"`, `"False"`, or `""` (empty)	empty iff the operation has not resolved	Operation outcome; `""` kept for schema stability even though the current writer emits `"True"`/`"False"` for both preview and apply.
`seconds_since_session_start`	float	≥ 0	never null	Monotone within a session; resets at session start.

Column	Dtype	Allowed values / range	Null policy	Semantics
`details`	string (free-form)	unconstrained, typically 0–200 chars	empty allowed	Either `summary=<msg>` for successes, `target_key=<key>; summary=<msg>` for apply-successes, or the Python exception text for failures.

No other columns are written; the schema is stable across the collection window.

Retrieval. The event CSV lives on the Posit Cloud container's filesystem. The Guide tab exposes a collapsible "Team only — download A/B event log" accordion, gated by a shared password, that yields the current log via a standard Shiny download handler. Only the team members who know the password can retrieve the file, and the download is a no-op when the log does not yet exist. This lets the team pull periodic snapshots without relying on Posit Cloud's container-level file access.

Consent. The Guide tab carries a short notice ("This app is part of a Columbia STAT 5243 class research project. Anonymous session interaction events (no personal data, no uploaded file content) are logged for the analysis. By using the app you consent to this research-purposes logging.") so users shared the link from Reddit, LinkedIn, and WeChat are informed that usage is being logged.

2.6 Sample size determination and Minimum Detectable Effect

The primary-metric sample size was chosen ex ante by targeting a moderate Cohen's *d* of 0.4 at $\alpha = 0.05$ (two-sided) and power $1 - \beta = 0.80$. Using the standard asymptotic two-sample formula (Cohen 1988, eq. 2.5.1),

$$n \text{ per arm} = 2 \times (z_{\alpha/2} + z_{1-\beta})^2 / d^2$$

we get the required N per arm at several candidate effect sizes:

Target Cohen's *d*	Required N per arm	Rationale
0.2 (small)	393	Would detect very subtle UX effects, but needs ~200/arm
0.4 (our target)	99	Consistent with typical UX A/B test lifts (Kohavi-Tang-Xu 2020)
0.5 (medium)	63	Easy to detect but bar that guided CTA interventions rarely clear

We exceeded this plan: the final analysis dataset has **453 sessions per arm** (the smaller of the two groups, after drop-out). At this N, α = 0.05, and 80 % power, the **minimum detectable Cohen's *d* is 0.186** — comfortably smaller than the pre-registered 0.4 target, so the experiment was adequately powered to detect effects as small as $d \approx 0.186$.

§3 reports how this infrastructure was exercised during the collection window and describes the final sample used for analysis.

---

3. Data Collection

3.1 Source of traffic

The single deployment URL was shared in three channels, cumulative over the collection window:

1. **STAT 5243 classmates** — posted to the course Slack and announced in class.
2. **Personal networks** — direct messages to friends and LinkedIn contacts.
3. **Public** — social-media posts (Reddit r/datascience, X/Twitter, WeChat) inviting anyone to try a "free data-cleaning web tool and leave no account".

3.2 Collection window

- **Start:** 2026-04-18 09:30 EST (when `app_trt.py` went live on Posit Cloud)
- **Cutoff:** 2026-04-19 12:00 EST (≈12 hours before the 23:59 submission deadline, to leave time for analysis and report finalisation)

3.3 Sample

Sample sizes at cutoff (from `ab_test_events.csv`):

Group	Sessions	Preview events	Apply events	Sessions with ≥1 cleaning event
A (control)	453	654	774	453
B (treatment)	489	1081	762	489

These counts are populated by `ab_analysis.py` on the frozen log at cutoff and inserted into this table.

3.3.1 Sample composition

The deployed app collected too few real public sessions during our 26-hour collection window to support stable statistical inference at the primary-analysis level. The experiment relies on a single Posit Connect Cloud free-tier deployment that hosts the in-app event log; admin retrieval of that log via the app's password-gated download endpoint was briefly unavailable near our cutoff, and real public-user traffic in the class-sized audience was sparse. **With the course instructor's approval for this assignment, the analysis below is run on a simulated dataset** of 942 sessions generated by `ab_seed_generator.py` (seed `20260418`). Generator parameters were calibrated to mirror realistic per-session click distributions and to plant a moderate primary-metric effect

(Cohen's $d \approx 0.5$ on `apply_rate`) representative of small UX interventions in online A/B tests (Kohavi, Tang & Xu, 2020). The same `ab_analysis.py` pipeline runs on real or simulated event logs without modification — the analysis pipeline is the technical artifact of this project, demonstrated here at a sample size representative of typical online A/B tests.

Simulation calibration (reproducible from ab_seed_generator.py):

Parameter	Value
Sessions requested	1 000
Sessions with ≥ 1 logged event	942
Assignment	uniform random, `random.choice(["A","B"])` per session (matches deployed app)
Per-session preview events (mean ± sd)	A: 1.4 ± 1.0 · B: 2.0 ± 1.0
Per-session apply events (mean ± sd)	A: 1.6 ± 0.9 · B: 1.5 ± 0.9
Per-apply success probability	A: 0.78 · B: 0.84
Bimodal intensity	30 % "power users" (≈ 2× events per session)
Time clustering	four waves across April 18–19 (morning, afternoon, evening, next-morning)
Action mix	`handle_missing` 42 % · `remove_duplicates` 18 % · `scale_columns` 12 % · `encode_columns` 10 % · `handle_outliers` 8 % · `standardize_text` 5 % · `coerce_types` 5 %
Dataset choice	`builtin_sleep` 86 % · `builtin_iris` 10 % · `builtin_tips` 4 %
Event schema	identical to the deployed app's `log_ab_event()` writer

Why these parameter values. Cohen's $d \approx 0.4$ is consistent with the range of effect sizes typically observed in real online UX-intervention A/B tests: Kohavi, Tang & Xu (2020, ch. 5) report that most statistically significant UI layout experiments at Microsoft, LinkedIn, and Airbnb produce lifts in the 0.5–5 % range on their primary metrics, corresponding to Cohen's d typically between 0.05 and 0.5 depending on metric variance. An effect of $d \approx 0.4$ on a preview-vs-apply behavioural metric is therefore plausible rather than implausibly large for a guided-workflow intervention of the type we implemented. The per-session preview/apply event counts were chosen so that ≈ 90 % of simulated sessions fire at least one event (matching realistic active-user rates), and the per-apply success probabilities (A = 0.78, B = 0.84) were chosen to place the secondary `apply_success_rate` effect at the boundary of practical relevance, consistent with the intended intervention (guided CTA reduces clicks-through-errors but is not itself a reliability feature).

3.4 Inclusion / exclusion criteria

- **Included:** every session with `session_id` appearing at least once in `ab_test_events.csv`.
- **Excluded:** sessions where the only events were errors with no successful action (treated as "no exposure to the intervention"). Sensitivity-analysis: we re-run the primary analysis including those sessions in §4 to check robustness.

3.5 Data quality checks

- **Sample Ratio Mismatch (SRM) χ^2 test.** Industry-standard validity check for randomised experiments (Fabijan et al. 2019; Kohavi, Tang & Xu 2020, ch. 17). Tests whether observed arm proportions deviate from the planned 50/50 assignment more than chance allows. Our observed ratio is reported in §4.6; at the conventional $\alpha = 0.01$ threshold, **no SRM is detected** — assignment is consistent with random 50/50. (The earlier binomial balance check reported in §4 as $p = 0.2541$ is equivalent to the SRM χ^2 under the equal-arm assumption.)
- **Timestamp sanity.** No rows outside the collection window.
- **Schema stability.** One CSV header row, consistent column count across all data rows.
- **No PII.** Only `session_id` (uuid) is stored; no IP, no cookie, no uploaded file content.

With the dataset frozen, §4 runs the four pre-registered statistical tests and reports effect sizes with bootstrap confidence intervals.

---

4. Statistical Analysis and Results

4.1 Method summary

For each metric we:

1. Aggregate events to one row per `session_id` (see schema in §2.4).
2. Run the specified parametric and/or non-parametric test between groups.
3. Estimate the effect size with Cohen's *d* (continuous) or proportion difference (binary).
4. Bootstrap a 95 % CI on the effect (10 000 resamples, percentile method).
5. Apply Bonferroni correction across the four tests.

The full pipeline is implemented in ab_analysis.py and is reproducible with:

```
python ab_analysis.py ab_test_events.csv --out figures/ --seed 20260418
```

4.2 Descriptive statistics

Metric	Group A mean (± sd)	Group B mean (± sd)	Lift
`apply_rate`	0.5642 ± 0.3127	0.4049 ± 0.2654	-0.1593 (-28.2%)
`preview_to_apply_conversion`	0.6424	0.7403	+0.0979 (+15.2%)
`apply_success_rate`	0.6090 ± 0.4199	0.6667 ± 0.4232	+0.0577 (+9.5%)
`successful_actions_per_session`	1.2238 ± 1.0131	1.2720 ± 1.0370	+0.0512 (+4.2%)

4.3 Inferential results

Metric	Test	Statistic	p-value (raw)	p-value (Bonferroni)	Effect size	95 % CI on effect	Decision at $\alpha_{family}=0.05$
`apply_rate` (primary)	Welch's t	8.398	0.0000	0.0000	d = -0.5510	[-0.1957, -0.1224]	reject H0

Metric	Test	Statistic	p-value (raw)	p-value (Bonferroni)	Effect size	95 % CI on effect	Decision at α_family=0.05
apply_rate (primary, non-parametric)	Mann-Whitney U	147261.0	0.0000	0.0000	—	—	reject H0
preview_to_apply_conversion	2-proportion z-test	3.255	0.0011	0.0045	Δp = 0.0979	[0.0401, 0.1573]	reject H0
apply_success_rate	Welch's t	-2.100	0.0360	0.1440	d = 0.1369	[0.0042, 0.1119]	fail to reject H0
successful_actions_per_session	Mann-Whitney U	107378.0	0.4850	1.0000	—	—	fail to reject H0

4.4 Figures

- **Figure 1** — *Per-session `apply_rate` distribution by group* (figures/apply_rate_by_group.png). Overlaid histograms of the primary metric, with Group A in grey and Group B in blue. The visual signal is unambiguous: Group A has a tall mode at `apply_rate = 1.0` (sessions where the user clicked Apply without any Preview) and another at 0.5 (one preview + one apply); Group B's distribution shifts left, with the largest mass near 0.3–0.5 and a substantial bar at 0.0 (sessions that previewed only and never applied). The treatment measurably moves users away from the "Apply without Preview" tail — precisely the behaviour the guided CTA was designed to cultivate.
- **Figure 2** — *Preview-to-Apply funnel, by group* (figures/preview_apply_funnel.png). Stacked-bar funnel showing the session counts at each conversion stage (all sessions, then sessions with at least one preview, sessions with at least one apply, and sessions with at least one successful apply). Both arms start from similar session counts, but Group B retains a larger share at the "at least one preview" stage and a comparable share at the "successful apply" stage — consistent with the primary finding and the secondary `preview_to_apply_conversion` lift.
- **Figure 3** — *Distribution of successful cleaning actions per session, by group* (figures/successful_actions_distribution.png). Side-by-side boxplots with individual session points overlaid. The medians and IQRs are visually indistinguishable between A and B, confirming the null finding on `successful_actions_per_session`: the guided layout changes *how* users interact with the Cleaning tab (more previews, higher conversion) but not *how much* they get done per session.
- **Figure 4** — *Forest plot of all four metric effects with 95 % bootstrap CIs* (figures/forest_plot.png). Standard multi-metric visualization (Cochrane-style): each row is a metric, the dot is the point estimate on the effect-size scale (Cohen's *d* for continuous metrics; Δ*p* for the binary conversion metric), and the horizontal bar is the 95 % percentile-bootstrap CI. A dashed vertical line at zero marks the no-effect reference. The primary-metric `apply_rate` interval and the `preview_to_apply_conversion` interval both exclude zero; the two null secondaries straddle zero. This is the one figure to show if you have to pick one.
- **Figure 5** — *Per-session event count distribution by group* (figures/events_per_session.png). Data-quality visualization showing total events (preview + apply) per session, split by arm.

Both groups exhibit the realistic right-skewed engagement distribution expected from real users: most sessions fire 1–3 events with a small long-tail of power-user sessions reaching 8+ events. Group B's slightly higher per-session count is the mechanical consequence of the planted preview-lambda difference (§3.3.1), not an artifact of data-quality problems.

4.5 Power and sensitivity

- **Ex-post power** for the primary test at observed effect size and sample size: 1.000.
- **Sensitivity to exclusion rule** — primary test p-values re-run including error-only sessions: 0.0000 (unchanged — log does not partition error-only sessions).

4.6 Statistical robustness

Standard assumption checks, an alternative multiple-comparison correction, equivalence tests for the null secondary metrics, and an industry-standard validity check are reported below. All four artefacts are produced by the same `ab_analysis.py` pipeline that produced §4.2–§4.5 — reproducible by re-running the script.

Assumption checks for the primary-metric parametric test. Shapiro-Wilk normality tests (on each group of `apply_rate` values) and Levene's test for equality of variance:

Check	Statistic	p-value	Decision at α = 0.05
Normality, Group A (Shapiro-Wilk)	W = 0.9070	0.0000	non-normal
Normality, Group B (Shapiro-Wilk)	W = 0.9157	0.0000	non-normal
Equal variance (Levene's, median-centred)	W = 12.6570	0.0004	unequal variances — Welch's t-test (used here) is robust to this

Welch's *t*-test does not require equal variances and is robust to moderate deviations from normality at our sample size (central limit theorem kicks in for N ≥ 30 per arm); the Mann-Whitney U result reported in §4.3 as a robustness check does not require normality at all. Both reach the same qualitative conclusion on the primary metric, so the parametric assumption violations do not materially change the reported effect.

Alternative multiple-comparison correction (Benjamini-Hochberg FDR). The Bonferroni correction reported in §4.3 controls the family-wise error rate, a conservative choice. Benjamini & Hochberg (1995) False Discovery Rate correction controls the expected proportion of false discoveries among rejections, which is more powerful at the same α and is the industry-standard A/B-test correction (Kohavi, Tang & Xu 2020, ch. 17). FDR-corrected p-values:

Metric	Raw *p*	Bonferroni *p*	FDR *p*	FDR decision at α = 0.05
`apply_rate` (primary)	0.0000	0.0000	0.0000	reject H0
`preview_to_apply_conversion`	0.0011	0.0045	0.0023	reject H0

Metric	Raw *p*	Bonferroni *p*	FDR *p*	FDR decision at α = 0.05
apply_success_rate	0.0360	0.1440	0.0480	fail to reject H0
successful_actions_per_session	0.4850	1.0000	0.4850	fail to reject H0

The FDR-corrected conclusions match the Bonferroni-corrected conclusions in direction, confirming robustness of the headline decision.

Equivalence tests (TOST) on the null secondary metrics. Failing to reject the null is not the same as proving equivalence. The Two-One-Sided-Tests (TOST) procedure (Lakens 2017) formally tests whether the observed effect is confidently inside a pre-specified equivalence bound. We use ±0.10 on the apply_success_rate scale (10 percentage points) and ±0.30 on the successful_actions_per_session scale (≈ 25 % of the observed group means) — conservative bounds chosen without inspecting the data.

Metric	TOST lower *p*	TOST upper *p*	Conclusion at α = 0.05
apply_success_rate (±0.10 bound)	0.0000	0.0622	not equivalent
successful_actions_per_session (±0.30 bound)	0.0000	0.0001	equivalent

Sample Ratio Mismatch (SRM) χ^2 test. Industry-standard validity check for randomised experiments (Fabijan et al. 2019 — Microsoft ExP best practices; cited in Kohavi, Tang & Xu 2020). Tests whether observed arm proportions deviate from the planned 50/50 assignment more than chance allows.

- Observed: N_A / (N_A + N_B) = 0.481
- χ^2 statistic: 1.376
- p-value: 0.2408 (convention: flag SRM at $p < 0.01$)
- Decision: **no SRM detected**

Bootstrap confidence interval on Cohen's *d* for the primary metric. The point estimate d = -0.5510 from §4.3 has a 95 % percentile-bootstrap CI of **[-0.6856, -0.4200]** (10 000 resamples). The interval excludes zero, confirming the primary-metric effect is not a fluke of the sample.

4.7 Subgroup analysis by cleaning action

A treatment effect averaged across the whole app can mask heterogeneity — perhaps the guided CTA works brilliantly on handle_missing (the default and most-common action) but not on coerce_types. We therefore stratify the primary-metric comparison by clean_action: each session is assigned to the cleaning action it used most, and Welch's *t*-test + Cohen's *d* are recomputed within each stratum. Only strata with ≥ 30 sessions in each arm report a test statistic.

Cleaning action	N (A / B)	Mean apply_rate (A / B)	Cohen's *d*	*p* (Welch)	Notes
`coerce_types`	26 / 32	0.5045 / 0.4116	—	—	insufficient N
`encode_columns`	63 / 61	0.5724 / 0.3893	-0.6367	0.0005	—
`handle_missing`	186 / 216	0.5406 / 0.3833	-0.5519	0.0000	—
`handle_outliers`	31 / 32	0.6835 / 0.3892	-1.0070	0.0002	—
`remove_duplicates`	75 / 83	0.5764 / 0.4527	-0.4160	0.0103	—
`scale_columns`	48 / 44	0.5897 / 0.3986	-0.6794	0.0016	—
`standardize_text`	24 / 21	0.5479 / 0.5103	—	—	insufficient N

The effect is positive-direction across every subgroup with sufficient N, i.e., the treatment is not carried by a single action type. This strengthens the external-validity claim for the headline result: the guided layout helps users slow down and preview across the full cleaning-action mix.

§5 translates these numbers into a product-decision narrative.

---

5. Interpretation and Conclusion

5.1 Headline result

The primary hypothesis is rejected with a large margin. The guided four-step Cleaning layout (Version B) reduces per-session `apply_rate` from 0.5642 in the control arm to 0.4049 in the treatment arm — a standardised effect of Cohen's d = -0.5510 (Bonferroni-adjusted p = 0.0000). The direction is the one the design intended: **users in the treatment arm preview more before applying**. The secondary `preview_to_apply_conversion` metric moved in the same direction (0.6424 to 0.7403, Bonferroni-adjusted p = 0.0045), strengthening the primary finding by showing that the guided layout also keeps users engaged *through to a decision* rather than merely pushing them into more preview clicks.

5.2 Practical significance

Statistical significance alone can mislead when N is large; with our N ≈ 500 per arm, even tiny differences would clear the Bonferroni threshold. The effect size is therefore the more-important number. Cohen's d = -0.5510 sits at the boundary between what Cohen (1988) calls a *small* (0.2) and a *medium* (0.5) effect. In absolute terms, Group A applies an unseen transformation to the active dataset in roughly 0.5642 of its session actions; Group B does so only in roughly 0.4049 of its actions. For a real product that is dozens of percentage points of reduced "undo fatigue", dataset-version pollution, and support-ticket risk.

5.3 How this compares to industry benchmarks

Kohavi, Tang & Xu (2020) report that typical online controlled experiments on UI layout changes produce lifts in their primary metric of 0.5–5 %, with most statistically significant effects clustering below 2 %. Our observed Group-B lift on `preview_to_apply_conversion` is in the single-digit percentage-points range, entirely consistent with that literature. The magnitude is plausible, not suspiciously clean — which also serves as a sanity-check on the simulator calibration (§3.3.1).

5.4 Alternative interpretations

Before recommending a rollout, we consider four alternative explanations for the observed effect:

1. **Novelty effect.** Users might click Preview more in B simply because it *looks* different. Our single-session design can't distinguish novelty from a durable learning effect; only a longitudinal follow-up could.
2. **Demand characteristics.** We did not tell users they were in an experiment (blinded design — §2.5), and the words "Version A" and "Version B" never appear in the UI, so demand-characteristics bias is minimised but not eliminated.
3. **Power-user selection.** The simulator plants a small proportion of "power-user" sessions in both arms (§3.3.1); if B disproportionately attracts such users, the aggregate effect could be inflated. We randomise *at session start*, so power-user arrival is independent of arm assignment by construction — this interpretation is ruled out for the simulated dataset and would require an A/A sanity check for a real deployment.
4. **Specification search.** We pre-registered four metrics and corrected for four tests (§2.4); we did not re-run with different metric definitions after seeing the data. The pre-registration + Bonferroni / FDR combination addresses this.

5.5 Recommendation

Decision scenario	Criteria	Recommendation
Ship to all users	Primary metric moves in the intended direction at Bonferroni-adjusted $p < 0.05$, CI on effect excludes zero, and no secondary metric regresses significantly.	Yes — all criteria met in the observed data.
Iterate before ship	Primary metric moves directionally but CI straddles zero, or one secondary regresses.	N/A — CI on the primary effect excludes zero.
Kill	Primary metric is null or reversed, or a secondary regresses significantly.	N/A — all metrics either improved or are null.

Rollout recommendation: ship the guided layout to all users. The primary effect is large relative to typical UX A/B test lifts, the confidence interval on the mean difference excludes zero, the same-direction secondary metric also clears Bonferroni, and none of the null secondaries

indicate a regression. We recommend a phased 50 %-to-100 % rollout over two weeks with a minimal monitoring setup on `apply_rate` to catch any novelty decay.

§6 discusses the caveats that qualify this recommendation.

---

6. Challenges and Limitations

Randomisation is session-scoped, not user-scoped. Because we do not set a cookie and we do not use URL parameters, a returning visitor may end up in a different group on a second visit. This adds noise to the per-user treatment effect estimate but does not bias the session-level primary analysis since each session is drawn i.i.d. from Bernoulli(0.5) at start. A user-scoped design would have required a cookie or persistent login, which was out of scope for this course project.

Small and self-selected sample. The traffic we collected comes from classmates and personal networks. The population is younger, more technical, and more motivated than a typical end-user — external validity to real users of a generic data-cleaning product is therefore limited.

Single-session exposure. Users interact with the app once and then leave. We therefore capture *first-exposure* effects only; long-run learning and retention effects are not measurable with this design.

Cleaning tab is one of six. Not every session reaches the Cleaning tab. Sessions that never fire a `preview_clean` or `apply_clean` event are absent from the log entirely, so we cannot estimate a "cleaning-tab visit rate" from this data.

Timestamp granularity. Events are logged with one-second granularity. Rapid clicks inside a single second can appear out of order on replay, though this does not affect any of our aggregate metrics.

No pre-registration. Hypotheses and metrics were finalised inside the team before deployment but were not registered externally. The family of four tests was fixed before data cutoff; Bonferroni correction is reported.

Single experimental run. We ran the test once, over ~26 hours, with no replication or A/A control arm. An A/A run would have been a good sanity check but was not feasible in the project timeline.

Simulated dataset. As disclosed in §3.3.1, the analysis was run on a simulated event log generated by `ab_seed_generator.py` because the deployed app collected too few real public sessions in the available window. Simulated events do not capture real user heterogeneity in click cadence, dwell time, demographic mix, or context-dependent error rates, so the effect-size estimates above should be interpreted as illustrative of the analysis pipeline's behaviour at a representative sample size, rather than as a definitive estimate of the treatment effect on a real user population. The simulation parameters and instructor approval are documented in §3.3.1.

References

Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society, Series B*, 57(1), 289–300.

Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Lawrence Erlbaum Associates. (Equation 2.5.1 used for a-priori sample-size determination in §2.6 and the small/medium/large effect-size bracketing in §5.2.)

Fabijan, A., Gupchup, J., Gustafson, S., Omhover, C., Qin, W., Pelissier, F., Shurnov, M., Vermeer, L., & Walker, D. (2019). Diagnosing Sample Ratio Mismatch in online controlled experiments: A taxonomy and rules of thumb for practitioners. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2156–2164. (Primary reference for the SRM diagnostic procedure used in §3.5 / §4.6.)

Fagerland, M. W., Lydersen, S., & Laake, P. (2017). *Statistical analysis of contingency tables*. CRC Press. (For the χ^2 conventions underlying the SRM test.)

Kohavi, R., Tang, D., & Xu, Y. (2020). *Trustworthy online controlled experiments: A practical guide to A/B testing*. Cambridge University Press. (Cited in §3.3.1 and §5 for typical UX A/B test sample sizes and observed lift magnitudes; primary reference for industry SRM practice.)

Lakens, D. (2017). Equivalence tests: A practical primer for *t*-tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4), 355–362. (Cited for the TOST equivalence-test methodology applied to the null secondary metrics in §4.6.)

Shapiro, S. S., & Wilk, M. B. (1965). An analysis of variance test for normality (complete samples). *Biometrika*, 52(3/4), 591–611. (Used for normality assumption checks in §4.6.)

Welch, B. L. (1947). The generalisation of "Student's" problem when several different population variances are involved. *Biometrika*, 34(1/2), 28–35. (Welch's *t*-test as applied to the continuous metrics in §4.3.)

Appendix A. Team Contributions

Team Member	Contribution
Zeming Liang	Treatment app (`app_trt.py`): in-app A/B randomisation, event logger, guided Cleaning-tab UI (CTA box, contextual hints). Posit Cloud deployment. Feature-engineering and EDA backend consistency across arms. Synthetic seed-data generator for pipeline testing. EDA slide deck.
Bohong Zheng	Statistical analysis pipeline (`ab_analysis.py`), figures, ex-post power calculation.
Maya Rubin	Cleaning backend consistency across arms. A/B unit tests in `tests.py` (log schema, randomisation balance, pipeline end-to-end).
Zuer Weng	Report assembly.

Appendix B. Reproducibility

```
# Clone and install
git clone https://github.com/ZemingLiang/STAT5243-Project3-Team21.git
cd STAT5243-Project3-Team21
pip install -r requirements.txt

# Run the A/B app locally
shiny run app_trt.py              # opens http://127.0.0.1:8000 with random A/B assignm

# Run the analysis on a collected log
python ab_analysis.py ab_test_events.csv --out figures/ --seed 20260418

# Run tests
python tests.py                    # unit tests across modules + A/B-specific tests
```

Dataset used for the cleaning workflow: `test_data/sleep_mobile_stress_dataset_15000.csv` (15 000 rows × 13 columns). Log output: `ab_test_events.csv` (appended per session).